

Procesamiento y Análisis de Datos Masivos sobre MovieLens 20M

Aplicación de PySpark, MinHashing, LSH y FP-Growth en Databricks

Camila, Marcelo, Cristian. | Curso de Data Mining | UTEC, 2026-I

7 de mayo de 2026

Resumen. Aplicamos cuatro técnicas de minería de datos masivos sobre el dataset MovieLens 20M (20 000 263 ratings, 138 493 usuarios, 26 744 películas) en un entorno Databricks/PySpark con Photon. La API DataFrame supera a la API RDD por un factor de $17,4 \times$ en la operación de top-K. La aproximación de Jaccard mediante MinHash con $K=128$ alcanza un MAE de 0,018 y una correlación de Pearson de 0,88 en el estrato denso, prácticamente sin pérdida de capacidad de ranking. El barrido de bandas en LSH revela un compromiso explícito entre precisión y recall: la configuración ($b=16, r=8$) maximiza F1 para umbral 0,5, mientras que ($b=32, r=4$) lo hace para umbral 0,3. FP-Growth descubre 2 234 207 reglas con lift de hasta 9,22, dominadas por sagas cinematográficas. Los resultados muestran que el paradigma *retrieval barato + re-ranking caro* usado por plataformas como Netflix es directamente reproducible a escala MovieLens con las técnicas estudiadas.

1 Metodología

1.1 Plataforma y datos

Toda la ejecución se realizó en Azure Databricks con Runtime 18.2 ML (Spark 4.1.0, Scala 2.13) y aceleración Photon habilitada. El clúster utilizó workers tipo `Standard_EC8ads_v5` (Azure Confidential Compute, procesador AMD EPYC, 64 GB RAM, 8 cores) con autoescalamiento entre 1 y 4 workers según demanda y terminación automática tras 80 minutos de inactividad. El dataset MovieLens 20M [1] contiene 20 000 263 calificaciones con escala continua 0,5 a 5,0 sobre 26 744 películas por 138 493 usuarios. Solo se utilizaron los archivos `ratings.csv` y `movies.csv`; los archivos de *tags* y *genome* no fueron requeridos por la representación elegida en la Parte III.

1.2 Preprocesamiento

La limpieza incluyó eliminación defensiva de nulos y duplicados (no se detectaron, lo que confirma la calidad del dataset MovieLens), normalización de tres etiquetas heterogéneas de género (`Sci-Fi` → `Science Fiction`, `Film-Noir` → `Film Noir`, (`no genres listed`) → `Unspecified`), extracción del año desde el título y binarización del rating mediante el umbral $\tau=3.5$. La elección del umbral se justifica como punto medio entre la mediana muestral (3,5) y la media (3,53), equivalente al cuartil superior de la distribución; produce un balance 61 % *likes* y 39 % *dislikes*, suficientemente equilibrado para preservar señal en ambas clases.

1.3 Estrategia de procesamiento Spark vs. local

Toda agregación pesada (`groupBy`, `count`, `array_intersect`, hashes MinHash, FP-Growth) corre en Spark distribuido; la visualización y el post-procesamiento de resultados reducidos (top-N, traducción de IDs) corren en pandas local sobre el driver. Justificación cuantitativa en Sección 4.

1.4 Decisiones específicas de implementación

En la Parte III se eligió la **Opción A** (cada película se representa como el conjunto de usuarios que la calificaron positivamente) y, para MinHash y LSH, el camino **(a) implementación manual** sobre Spark en lugar de `MinHashLSH` de MLlib (camino b) o de combinación con MLlib (camino c). La elección no es por preferencia: la API de MLlib solo expone `numHashTables` y no b y r por separado, lo que impide el barrido $b \cdot r = K$ exigido por el enunciado; cada `fit()` con K distinto genera nuevas funciones hash, contaminando la comparación de configuraciones; `approxSimilarityJoin` fusiona la generación de candidatos con el filtrado por umbral, impidiendo medir TP, FP y FN aislados; y exige convertir

cada conjunto a **SparseVector** con dimensión igual al universo de usuarios, lo que satura el *broadcast* a escala. Los cuatro requisitos cuantitativos del enunciado de la Parte 3.3 (variar b y r con $b \cdot r$ constante, mantener las mismas firmas base al barrer K , separar candidatos LSH del filtrado por umbral, y comparar con el umbral teórico $s^*=(1/b)^{1/r}$) solo son satisfechos por la implementación manual. La opción (c) no aporta información adicional: la versión MLlib produciría un único punto del espacio (b, r) ya cubierto por nuestra configuración $(128, 1)$.

2 Hallazgos del Análisis Exploratorio (EDA)

2.1 Distribución de calificaciones y binarización

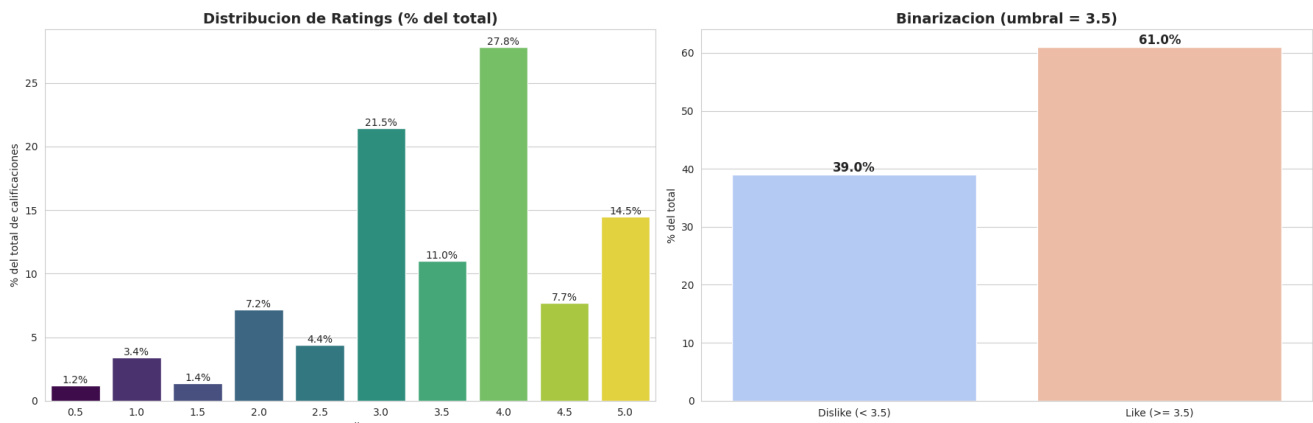


Figura 1: Distribución de calificaciones. Izquierda: porcentaje del total por valor de rating en escala continua. Derecha: distribución binarizada con umbral $\tau=3.5$ (*like/dislike*). El rating 4,0 concentra el 27,8 % de las observaciones; los valores enteros (1, 2, 3, 4, 5) acumulan el 65 % pese a estar disponible la escala fraccionaria.

La distribución (Figura 1) presenta sesgo positivo claro con masa concentrada entre 3,0 y 4,0. La preferencia humana por valores enteros queda evidente: 4,0 es la moda absoluta. La binarización con $\tau=3.5$ separa el cuartil superior y produce un balance 61/39, suficientemente equilibrado para evitar el problema de clase mayoritaria que tendría un umbral más bajo.

2.2 Actividad por usuario y por película

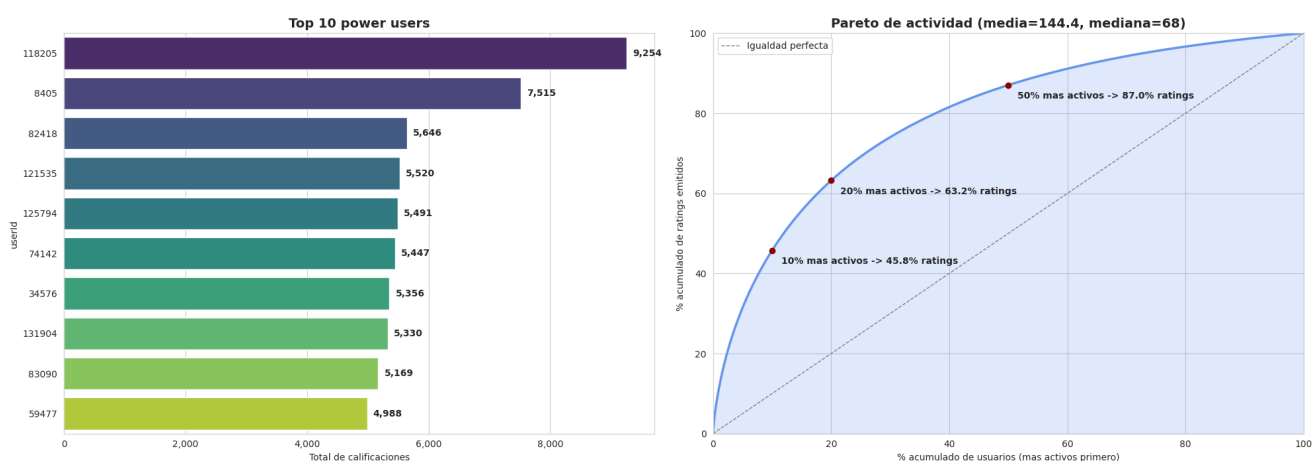
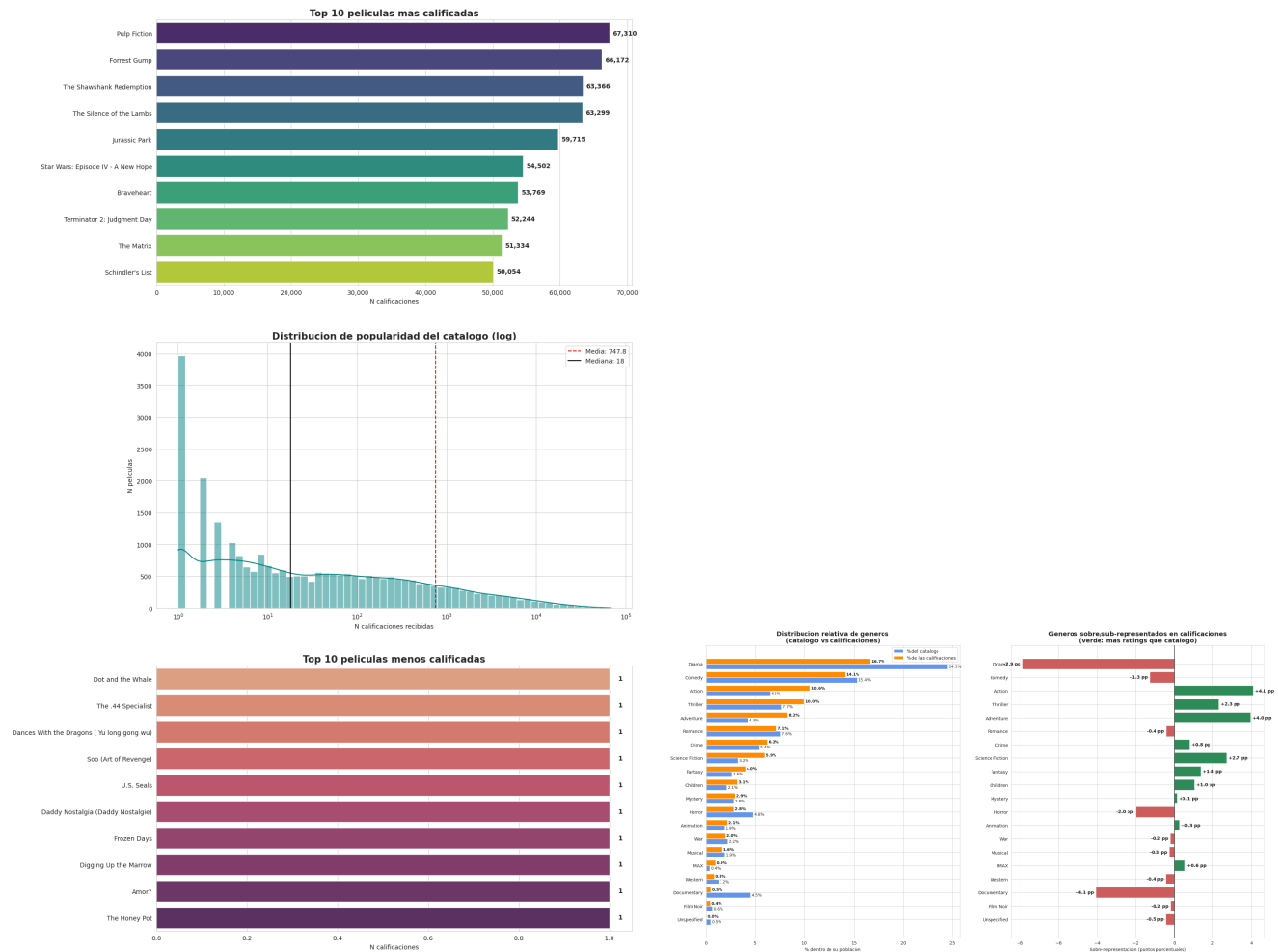


Figura 2: Actividad por usuario. Izquierda: top 10 usuarios con mayor número de calificaciones, donde el usuario más activo registra 9 254 ratings. Derecha: curva de Pareto de actividad, construida ordenando a los usuarios de mayor a menor número de calificaciones y acumulando su contribución al total de ratings. El 10 % más activo concentra aproximadamente el 45,8 % de las calificaciones, el 20 % más activo el 63,2 % y el 50 % más activo alrededor del 87,0 %.

La Figura 2 muestra que la actividad de los usuarios se encuentra fuertemente concentrada. Aunque MovieLens 20M impone un mínimo de 20 calificaciones por usuario, existe una brecha amplia entre el usuario típico y los usuarios hiperactivos: la mediana es 68, la media 144,4 y el máximo 9254. La diferencia entre media y mediana confirma un sesgo positivo: pocos usuarios generan una proporción desproporcionada de las interacciones. Esta concentración es metodológicamente relevante porque los usuarios más activos pueden dominar la señal agregada de similitud, especialmente cuando se construyen conjuntos de usuarios positivos por película.



(a) Actividad por película. El panel resume tres vistas complementarias: top 10 películas más calificadas, distribución de popularidad del catálogo y bottom 10 películas menos calificadas. La mediana es 18 ratings por película, mientras que la media alcanza 747,8; esta diferencia implica un ratio media/mediana cercano a 41 $\times$.

(b) Distribución relativa de géneros en el catálogo y en las calificaciones, junto con la diferencia porcentual entre ambas distribuciones. El gráfico permite identificar géneros sobrerepresentados en consumo efectivo frente a su disponibilidad en catálogo, así como géneros que aparecen con alta presencia en catálogo pero menor peso relativo en ratings.

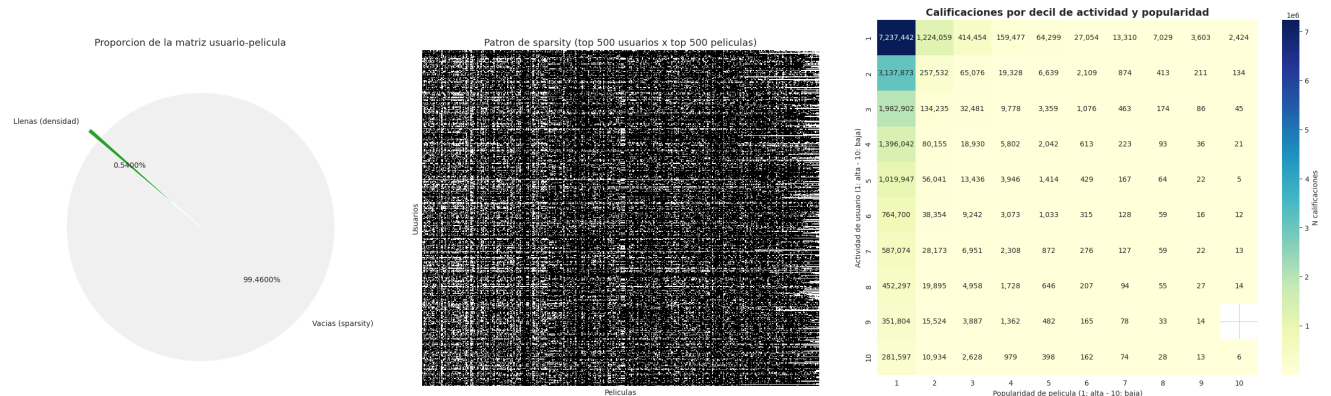
Figura 3: Evidencia de concentración en MovieLens 20M. La actividad por película (a) presenta una cola larga extrema: pocas películas concentran decenas de miles de ratings, mientras muchas tienen evidencia mínima. El contraste por género (b) muestra que la popularidad observada no depende solo de la disponibilidad de películas en el catálogo, sino también de la intensidad con que ciertos géneros son efectivamente calificados por los usuarios.

La Figura 3 evidencia que la concentración por película es incluso más severa que la concentración por usuario. Mientras películas masivas como *Pulp Fiction*, *Forrest Gump* o *The Shawshank Redemption* acumulan decenas de miles de calificaciones, una parte considerable del catálogo tiene muy poca evidencia empírica. Esta estructura de cola larga implica que muchas películas no cuentan con suficientes usuarios en común para producir estimaciones estables de similitud. En particular, las películas con una o muy pocas calificaciones pueden generar valores de Jaccard artificialmente altos o bajos por pura escasez de datos.

Por ello, el corte de 50 *likes* mínimos por película en la Parte III está justificado desde el EDA: elimina

ítems con evidencia insuficiente antes de aplicar Jaccard, MinHash y LSH. Sin este filtro, la comparación entre películas estaría contaminada por ruido estadístico procedente de ítems casi no observados. Además, el contraste de géneros en la Figura 3b sugiere que la actividad no solo responde a cuántas películas existen en cada género, sino también a qué géneros reciben más interacción efectiva por parte de los usuarios. En conjunto, las Figuras 2 y 3 confirman un patrón típico de sistemas de recomendación: usuarios e ítems siguen dinámicas de tipo *power-law* o *rich-get-richer*, donde una minoría concentra gran parte de las interacciones observadas.

2.3 Distribución de géneros y densidad de la matriz



(a) Comparación porcentual del peso de cada género en el catálogo y en las calificaciones. Drama domina ambos espacios, pero su peso relativo disminuye al pasar de disponibilidad en catálogo a consumo efectivo; en contraste, géneros como Action y Adventure ganan participación relativa en ratings.

(b) *Heatmap* de cobertura por deciles de actividad de usuario y de popularidad de película. Densidad global 0,54%; el cuadrante superior derecho concentra el 36% del total.

Figura 4: Distribución de géneros (a) y densidad de la matriz usuario $\times$ película (b). La diferencia entre catálogo y ratings indica que la disponibilidad de películas no se traduce linealmente en consumo observado. La concentración del 36% de los ratings en el 1% superior de celdas (top 10% usuarios $\times$ top 10% películas) motiva el muestreo estratificado en la Parte III.

La densidad global de la matriz usuario $\times$ película es del 0,54% (99,46% de *sparsity*), valor típico de sistemas de recomendación a esta escala. La visualización por deciles (Figura 4b) revela que la actividad no se distribuye uniformemente sobre el conjunto disperso: el cuadrante *top 10% usuarios $\times$ top 10% películas* concentra el 36% de todas las observaciones, una densidad local $67\times$ superior al promedio. Esta concentración tiene una consecuencia metodológica directa: la búsqueda de pares similares debe priorizar el régimen denso, donde efectivamente viven las similitudes informativas.

3 Resultados

3.1 Parte II: procesamiento distribuido y *benchmark* RDD vs. DataFrame

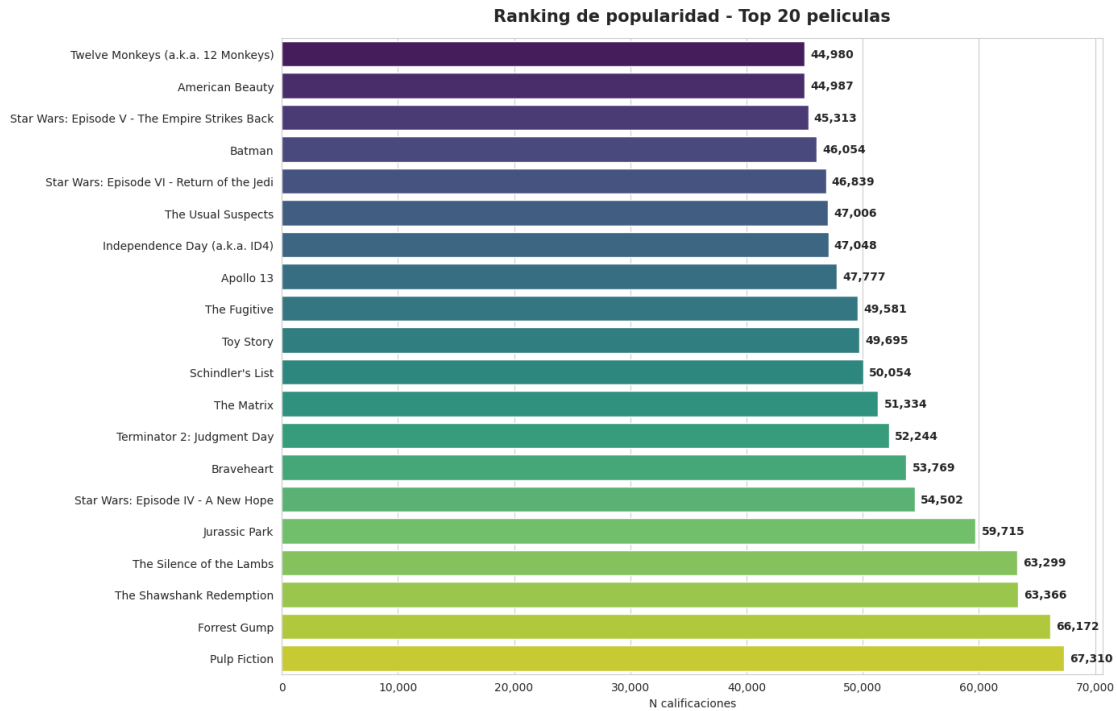


Figura 5: Top 20 películas por número de calificaciones, ordenadas ascendentemente para facilitar la lectura. Pulp Fiction lidera con 67 310 ratings, seguida de Forrest Gump (66 172) y The Shawshank Redemption (63 366). Los géneros Drama, Crime y Thriller dominan el top.

El *benchmark* entre las APIs RDD y DataFrame sobre la operación de top-K (Tabla 1) revela una diferencia mucho mayor que la típicamente reportada en literatura introductoria. El factor $17,4 \times$ se explica por la combinación de tres mecanismos: (i) Catalyst combina agregación, ordenamiento y proyección en un único plan físico optimizado; (ii) Tungsten mantiene los datos en formato columnar binario dentro de la JVM, evitando *boxing* de objetos; (iii) la API RDD requiere serialización Python $\leftrightarrow$ JVM por fila, costo amortizado de manera deficiente sobre 20 millones de operaciones. Ambas APIs producen idéntico top-3, validando la consistencia funcional.

Tabla 1: Tiempo de ejecución del cómputo de top-20 sobre 20 000 263 filas. Azure Databricks Runtime 18.2 ML, workers `Standard_EC8ads_v5` con Photon habilitado; variación entre ejecuciones $< 5\%$.

API	Tiempo (s)	Speedup	Top-3 obtenido
RDD (<code>map</code> $\rightarrow$ <code>reduceByKey</code> $\rightarrow$ <code>takeOrdered</code>)	31,08	1,0	296, 356, 318
DataFrame (<code>groupBy</code> $\rightarrow$ <code>orderBy</code> $\rightarrow$ <code>limit</code>)	1,79	17,4	296, 356, 318

3.2 Parte III: similitud, MinHash y LSH

Tras aplicar los cortes `min_likes_película=50` y `min_likes_usuario=20` el dataset operativo queda con 8 320 películas, 107 001 usuarios y 11 634 899 *likes* (densidad 1,31 %, aproximadamente $2,4 \times$ la densidad original). Sobre este universo se construyó una muestra estratificada de 507 689 pares: 499 500 densos (todos los $\binom{1000}{2}$ del top-1 000 de películas) y 8 189 aleatorios.

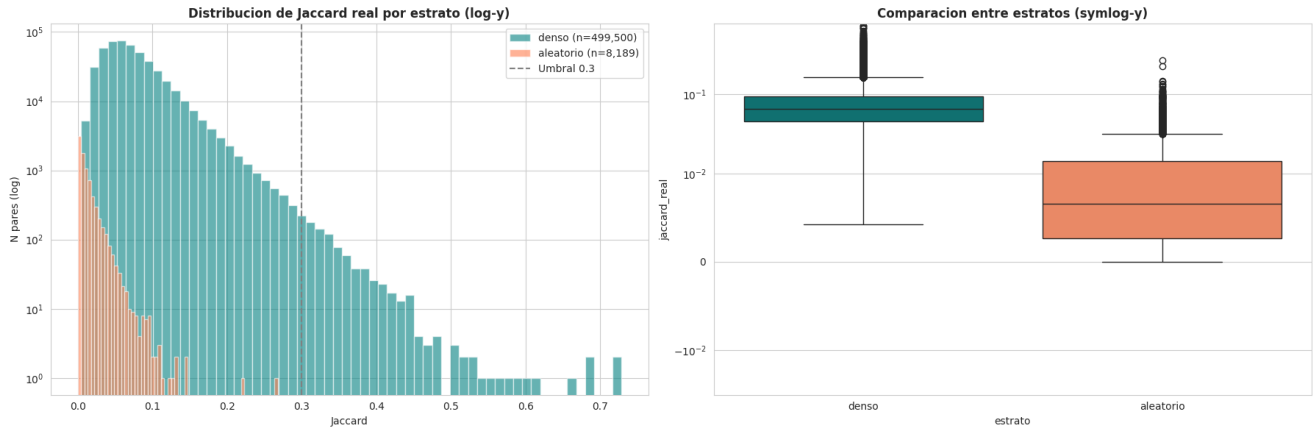
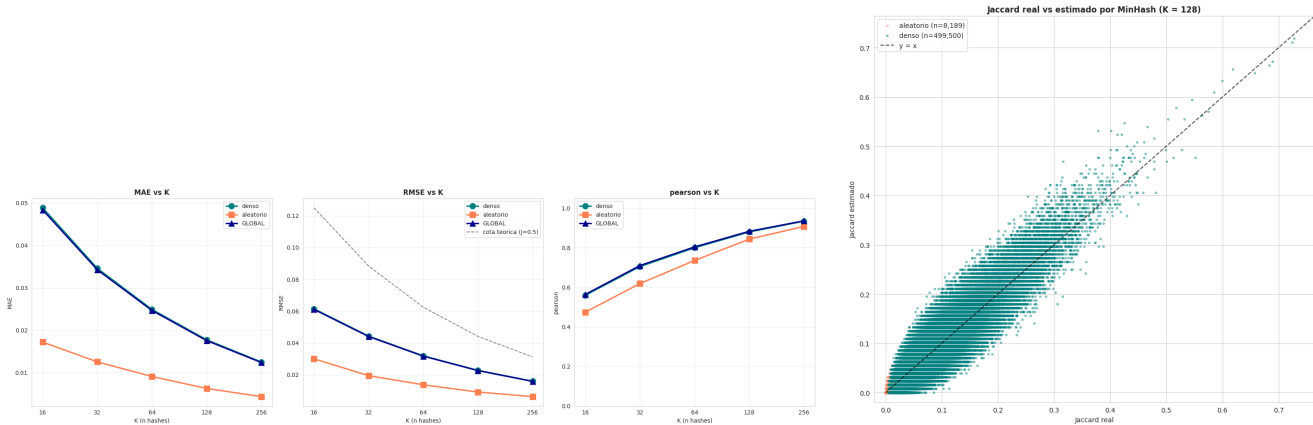


Figura 6: Distribución de Jaccard exacto por estrato. Izquierda: histograma con escala logarítmica en el eje vertical y línea vertical en el umbral $J=0.3$. Derecha: *boxplot* comparativo en escala simétrica logarítmica. El estrato denso (mediana 0,065, máximo 0,728) tiene una distribución sustancialmente más cargada hacia valores altos que el aleatorio (mediana 0,007). La separación valida la estratificación: los pares informativos viven en el régimen denso.

El cálculo de Jaccard exacto se ejecuta con `array_intersect` de Spark, que opera en JVM/Tungsten sin penalización de UDF Python. Los pares por umbral son: 888 con $J \geq 0.30$, 105 con $J \geq 0.40$ y 18 con $J \geq 0.50$. El par más similar es *The Lord of the Rings: The Fellowship of the Ring* vs. *The Two Towers* con $J=0.728$ (intersección 24 734 usuarios sobre unión 33 979). Los 20 pares más similares corresponden, sin excepción, a películas de la misma saga (LOTR, Star Wars, Harry Potter, Bourne, Indiana Jones, Padrino), lo que constituye una validación cualitativa fuerte del método.



(a) Métricas MinHash en función de K . MAE y RMSE decrecen siguiendo la cota teórica $\propto 1/\sqrt{K}$; Pearson sube monótonicamente.

(b) Dispersión Jaccard real vs. estimado con $K=128$. Los puntos se distribuyen simétricamente alrededor de $y=x$ sin sesgo sistemático.

Figura 7: Calidad de la aproximación MinHash. La gráfica (a) confirma empíricamente la decadencia $1/\sqrt{K}$: cuadruplicar K divide el MAE entre $\sqrt{4}=2$. La gráfica (b) muestra que el estimador es insesgado y que su varianza se reduce hacia los extremos $J=0$ y $J=1$, según la cota teórica $J(1-J)/K$.

La aproximación MinHash con $K=128$ funciones hash logra MAE de 0,018 y correlación de Pearson de 0,88 en el estrato denso (Figura 7), reduciendo el costo de comparación por par desde arrays de hasta 51 502 enteros (Shawshank) a tan solo 128, una mejora aproximada de $400 \times$.

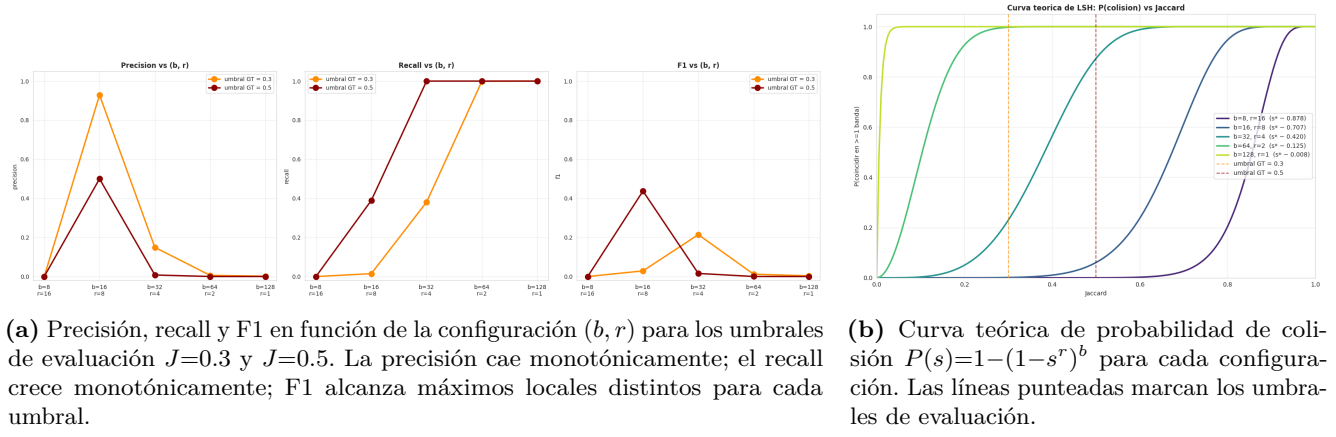


Figura 8: Trade-off de LSH con $K_{LSH}=128$ y $b \cdot r=128$. La inflexión empírica de la curva de recall en (a) coincide con el umbral teórico $s^*=(1/b)^{1/r}$ predicho por la curva en (b), validando la teoría sobre datos reales.

El barrido de bandas (Tabla 2) revela un trade-off explícito entre eficiencia computacional y calidad de los resultados, exactamente como predice la teoría LSH. La configuración $(b=8, r=16)$ es inviable porque $s^*=0.878$ supera el Jaccard máximo observado en el dataset (0,728); LSH no genera ningún candidato. La configuración $(b=16, r=8)$ es la óptima para detección de pares casi duplicados (sagas) con F1 de 0,438 en umbral 0,5, mientras que $(b=32, r=4)$ es óptima para descubrimiento amplio en umbral 0,3 (F1 de 0,214). Las configuraciones $(b=64, r=2)$ y $(b=128, r=1)$ saturan el recall pero generan 750 105 y aproximadamente $1,98 \times 10^7$ pares candidatos respectivamente, viables solo si existe un re-ranker downstream que filtre por similitud exacta.

Tabla 2: Barrido LSH manual con $K_{LSH}=128$ y $b \times r=128$. Las cinco configuraciones cubren el espacio completo desde el régimen ultra-conservador hasta el ultra-agresivo. Los mejores valores de F1 por umbral aparecen en **negrita**.

b	r	s^*	Candidatos	$J \geq 0.3$		$J \geq 0.5$	
				Precisión	F1	Precisión	F1
8	16	0,878	0	0,000	0,000	0,000	0,000
16	8	0,707	19	0,929	0,029	0,500	0,438
32	4	0,420	3 545	0,149	0,214	0,008	0,016
64	2	0,125	750 105	0,006	0,012	0,000	0,000
128	1	0,008	19 810 787	0,002	0,004	0,000	0,000

3.3 Parte IV: reglas de asociación con FP-Growth

Sobre las 107 001 transacciones (mediana 60 películas por usuario, máximo 3571), FP-Growth con minSupport=0.05 y minConfidence=0.5 entrenó en 6,8 segundos y descubrió 508 050 itemsets frecuentes ($k_{\max}=9$) y 2 234 207 reglas de asociación. Las reglas presentan confianza promedio 0,77 y *lift* promedio 2,42, con valores máximos de 0,985 y 9,22 respectivamente.



Figura 9: Visualización del conjunto completo de reglas de asociación. Izquierda: dispersión en el plano *soporte* vs. *confianza* con *lift* codificado en color (más amarillo = mayor *lift*). Derecha: top 10 reglas ordenadas por *lift*. Las reglas más informativas corresponden sin excepción a películas de la misma franquicia (Harry Potter, Bourne).

Tabla 3: Top 10 reglas de asociación por *lift*. Las cuatro primeras corresponden a películas de la misma saga; las restantes son cruces entre franquicias afines (Harry Potter $\times$ LOTR, Harry Potter $\times$ Shrek), patrones que no se infieren de los metadatos de género.

Antecedente	Consecuente	Soporte	Confianza	Lift
HP Azkaban + HP Sorcerer's Stone	HP Chamber of Secrets	0,052 8	0,834 1	9,219
HP Goblet of Fire	HP Prisoner of Azkaban	0,055 4	0,799 7	8,580
HP Prisoner of Azkaban	HP Goblet of Fire	0,055 4	0,594 4	8,580
HP Sorcerer + LOTR Two Towers + LOTR Fellowship	HP Chamber of Secrets	0,053 7	0,743 3	8,214
HP Chamber + HP Azkaban	HP Sorcerer's Stone	0,052 8	0,885 1	8,191
HP Sorcerer + LOTR Return of the King	HP Chamber of Secrets	0,051 3	0,735 4	8,127
HP Sorcerer + LOTR Two Towers	HP Chamber of Secrets	0,055 7	0,735 1	8,124
Bourne Ultimatum + Bourne Identity	Bourne Supremacy	0,053 0	0,799 9	7,929
HP Chamber + HP Sorcerer's Stone	HP Prisoner of Azkaban	0,052 8	0,737 4	7,912
HP Sorcerer + Shrek	HP Chamber of Secrets	0,051 4	0,704 9	7,790

Las reglas con mayor *lift* (Figura 9, Tabla 3) corresponden, casi sin excepción, a películas de la misma franquicia. Esto valida el algoritmo pero no constituye un descubrimiento comercial fuerte. El valor real está en las reglas *cruzadas entre franquicias distintas* (Harry Potter $\times$ LOTR con *lift* entre 7,7 y 8,2; Harry Potter $\times$ Shrek con *lift* 7,79), que revelan una audiencia con perfil específico de fantasía/familiar y que no se infiere desde los metadatos de género (cada franquicia ocupa categorías nominalmente distintas).

Mientras que el *lift* ordena las reglas por sorpresa estadística, la *confianza* y el *soporte* responden a preguntas distintas y complementarias: ¿qué reglas son más deterministas en su predicción? y ¿qué reglas aplican al mayor número de usuarios? Las Tablas 4 y 5 muestran los rankings correspondientes.

Tabla 4: Top 7 reglas de asociación por *confianza*. Los antecedentes son combinaciones largas (4 a 8 películas) que predicen con casi certeza el *like* a un blockbuster transversal (*The Godfather*, *LOTR: The Fellowship of the Ring*). El *lift* permanece en rango 3,08 a 3,41, un orden de magnitud por debajo del top por *lift*, reflejando el sesgo de la confianza hacia consecuentes con alto soporte marginal.

Antecedente	Consecuente	Soporte	Confianza	Lift
Goodfellas + Godfather II + Memento + Usual Suspects	The Godfather	0,052 9	0,985 0	3,080
Goodfellas + Godfather II + Memento + Matrix	The Godfather	0,050 4	0,984 7	3,079
Gladiator + LOTR ROTK + LOTR TT + Sixth Sense + SW VI + SW V + Matrix	LOTR Fellowship	0,051 3	0,984 2	3,412
Goodfellas + Godfather II + Se7en + Usual Suspects + Silence of the Lambs	The Godfather	0,051 0	0,984 1	3,077
Indiana Jones LC + Gladiator + LOTR ROTK + LOTR TT + SW VI + Raiders + SW V + SW IV	LOTR Fellowship	0,050 2	0,984 1	3,412
Shrek + Gladiator + LOTR ROTK + LOTR TT + SW V + SW IV + Matrix	LOTR Fellowship	0,051 2	0,984 0	3,412
Indiana Jones LC + Gladiator + LOTR ROTK + LOTR TT + Raiders + SW V + Matrix	LOTR Fellowship	0,052 5	0,983 9	3,411

Las reglas top por *confianza* (Tabla 4) revelan un patrón distinto al del top por *lift*: presentan antecedentes largos (entre 4 y 8 películas) y consecuentes que son blockbusters transversales con soporte marginal alto, lo que arrastra mecánicamente la confianza por encima de 0,98. El *lift* en este top se sitúa entre 3,08 y 3,41, valores ciertamente positivos pero un orden de magnitud por debajo de los 9,22 alcanzados en el ranking por *lift*. Esto ilustra la limitación clásica de la confianza como criterio único: una regla del tipo $A \Rightarrow B$ donde B ocurre en el 30 % del corpus tendrá confianza alta para casi cualquier A con soporte mínimo, sin que ello signifique que A aporte información sobre B . La métrica de *lift* corrige este sesgo dividiendo entre el soporte marginal de B , recuperando la noción de *independencia estadística*.

Tabla 5: Top 10 reglas de asociación por *soporte*. Las reglas que aplican al mayor número de usuarios involucran únicamente antecedentes y consecuentes individuales formados por blockbusters mainstream (*Pulp Fiction*, *Shawshank Redemption*, *The Silence of the Lambs*, *Forrest Gump*) o por la trilogía original de *Star Wars*. El *lift* cae al rango 1,33 a 2,15, próximo a la independencia pero todavía positivo.

Antecedente	Consecuente	Soporte	Confianza	Lift
Pulp Fiction	The Silence of the Lambs	0,305 6	0,660 1	1,457
The Silence of the Lambs	Pulp Fiction	0,305 6	0,674 6	1,457
Pulp Fiction	The Shawshank Redemption	0,305 5	0,659 7	1,371
The Shawshank Redemption	Pulp Fiction	0,305 5	0,634 6	1,371
The Silence of the Lambs	The Shawshank Redemption	0,302 4	0,667 6	1,387
The Shawshank Redemption	The Silence of the Lambs	0,302 4	0,628 4	1,387
SW: A New Hope	SW: The Empire Strikes Back	0,294 4	0,749 8	2,147
SW: The Empire Strikes Back	SW: A New Hope	0,294 4	0,842 8	2,147
The Shawshank Redemption	Forrest Gump	0,280 2	0,582 1	1,330
Forrest Gump	The Shawshank Redemption	0,280 2	0,640 2	1,330

El ranking por *soporte* (Tabla 5) está dominado por reglas con antecedente y consecuente de tamaño 1 (pares de blockbusters individuales) y captura el patrón de *co-consumo masivo* típico del cine mainstream: *Pulp Fiction* y *The Silence of the Lambs* co-aparecen positivamente en el 30,56 % de los 107 001 usuarios filtrados, valor que las convierte en el par con mayor co-ocurrencia bidireccional del corpus. La presencia de la trilogía original de *Star Wars* entre las top 10 con *lift* 2,15 es destacable: a pesar de que el ranking filtra por popularidad bruta, la afinidad intra-saga sigue dominando estadísticamente sobre la afinidad

entre dramas clásicos aislados (*Shawshank* $\leftrightarrow$ *Forrest Gump*, *lift* 1,33). Operativamente, estas reglas son las más *accionables* en producción porque se disparan tan pronto como el usuario expresa una sola preferencia, pero las menos *informativas*: recomendar *Shawshank* a alguien que vio *Pulp Fiction* solo aporta un 37 % de mejora respecto al *baseline* de recomendarla a un usuario aleatorio.

Distribución por longitud del antecedente. Las 2,2 M reglas se reparten asimétricamente según $k_{\text{ant}} \equiv |A|$: 5 624 con $k_{\text{ant}}=1$, 133 560 con $k_{\text{ant}}=2$, 551 327 con $k_{\text{ant}}=3$, máximo en 826 370 con $k_{\text{ant}}=4$, y caída posterior hasta 1 170 con $k_{\text{ant}}=8$. La distribución sigue el patrón de campana heredado de los itemsets frecuentes ($k_{\text{máx}}=9$, con pico en $k=5$). Para un sistema de recomendación en producción, las reglas con $k_{\text{ant}} \leq 2$ (139 184 reglas, 6,2 % del total) son las directamente accionables, ya que requieren observar a lo más dos preferencias previas del usuario antes de poder dispararse. Filtrando además por $\text{lift} \geq 3$ se obtiene un conjunto operativo manejable con buen balance entre cobertura y poder predictivo. Las 826 370 reglas con $k_{\text{ant}}=4$ son numéricamente dominantes pero exigen que el usuario haya calificado positivamente cuatro películas específicas para activarse, condición rara en el régimen *cold-start* típico de un nuevo suscriptor.

Caso ilustrativo: Matrix. El enunciado del proyecto sugería *Matrix* $\Rightarrow$ *Terminator 2* como ejemplo. Sobre las 2,2 M reglas extraídas, 589 132 contienen *The Matrix* en el antecedente y 144 500 en el consecuente. La regla con mayor confianza con *Matrix* como consecuente es *Matrix Reloaded* + *SW VI* + *SW V* $\Rightarrow$ *The Matrix* con confianza 0,978 y *lift* 2,46: un usuario que dio *like* a la secuela y a dos *Star Wars* clásicas tiene 97,8 % de probabilidad de haber dado *like* también a *Matrix*. La regla con mayor *lift* con *Matrix* en el antecedente es *HP Sorcerer's Stone* + *Matrix* $\Rightarrow$ *HP Chamber of Secrets* (*lift* 7,58), que pertenece al cluster cruzado fantasía/ciencia ficción identificado en el ranking por *lift*. La asimetría 589 K vs. 144 K refleja un hecho operativo: *Matrix* es más útil como *evidencia entrante* (una preferencia fuerte y bien observada del usuario) que como *objetivo* de recomendación, dado que su soporte marginal es tan alto que pocos antecedentes elevan su probabilidad sustancialmente por encima del *prior*.

4 Discusión Crítica

Pérdida de precisión por aproximación. MinHash sin LSH es prácticamente sin pérdida con $K=128$: MAE de 0,018 (1,8 puntos porcentuales sobre $[0, 1]$) y Pearson de 0,88, lo que conserva el ranking de pares. La pérdida real aparece con LSH y depende de (b, r) : (16, 8) es óptima para alta precisión y umbral estricto, (32, 4) para alta cobertura y umbral relajado; ninguna configuración domina, lo que obliga a definir explícitamente el caso de uso antes de fijar hiperparámetros.

Impacto de falsos positivos y negativos. Los FP (recomendaciones poco relacionadas) se mitigan con re-ranking exacto sobre los candidatos y una proporción controlada es deseable para diversidad y para evitar el *filter bubble*. Los FN (películas verdaderamente similares que LSH nunca propone) son invisibles desde la perspectiva del sistema, no admiten recuperación downstream y degradan acumulativamente la calidad percibida. Por estas asimetrías, la práctica industrial prefiere configuraciones de alto recall con re-ranker antes que alta precisión con recall pobre.

¿Spark valió la pena? Spark gana cuando el dato es distribuido y la operación es agregación o join sobre volúmenes que exceden la RAM de un nodo: lectura del CSV de 500 MB, `groupBy` distribuido, `array_intersect` en JVM, FP-Growth nativo. Pierde cuando se requiere lógica Python por fila (UDFs costosas) o cuando el intermedio cabe en RAM: la traducción de IDs y el ranking de 2,2 M reglas se ejecutan en pandas local $\sim 5 \times$ más rápido que el equivalente en Spark. La regla práctica: *agregar en Spark, presentar en pandas*.

Cuellos de botella de escalabilidad. *Memoria*: matriz densa $138\,493 \times 26\,744 \approx 3.7 \times 10^9$ celdas, inviable; la esparcida es esencial. *Tiempo*: RDD escala mal por serialización (31 s para top-K sobre 20 M filas); un dataset $10 \times$ mayor lo llevaría a ~ 5 min mientras DataFrame queda en 18 s. *Cardinalidad*: cross-join naive sobre 8 320 películas generaría $34,6 \times 10^6$ pares; sin estratificación y sin LSH, Jaccard exacto sería $\sim 70 \times$ más costoso.

Aplicabilidad en plataformas reales. Netflix ($\sim 250\,000$ títulos, 230 M usuarios) tiene un cross-join naive de $\binom{250\,000}{2} \approx 3.1 \times 10^{10}$ pares, prohibitivo; LSH lo reduce a $\sim 32 \times 10^6$ operaciones de hash ($\sim 1\,000 \times$). El paradigma *MinHash* + *LSH* para *retrieval*, *modelo neuronal para ranking* es el de producción y nuestro proyecto reproduce el primer eslabón. Spotify (~ 100 M canciones) sustituye Jaccard por embeddings

densos y LSH por *Approximate Nearest Neighbor* (FAISS, ScaNN, HNSW), pero el esquema persiste. Amazon usa reglas FP-Growth como base de *los clientes que compraron X también compraron Y*, con minSupport inversamente proporcional al tamaño del catálogo.

Conclusión

Las técnicas estudiadas producen resultados sólidos con ganancias sustanciales ($17 \times$ DataFrame vs RDD, $400 \times$ MinHash vs comparación exacta, $1\,000 \times$ LSH a escala industrial) y compromisos cuantificables. La arquitectura *retrieval barato + ranking caro* es reproducible con estas herramientas; el escalado a producción es cuestión de paralelismo y de reemplazar conjuntos por embeddings.

Referencias

- [1] Harper, F. M., Konstan, J. A. (2015). *The MovieLens Datasets: History and Context*. ACM TiiS, 5(4):1 a 19.
- [2] Broder, A. Z. (1997). *On the Resemblance and Containment of Documents*. Compression and Complexity of Sequences.
- [3] Indyk, P., Motwani, R. (1998). *Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality*. ACM STOC.
- [4] Han, J., Pei, J., Yin, Y. (2000). *Mining Frequent Patterns without Candidate Generation*. ACM SIGMOD.
- [5] Armbrust, M. et al. (2015). *Spark SQL: Relational Data Processing in Spark*. ACM SIGMOD.